



丸一口

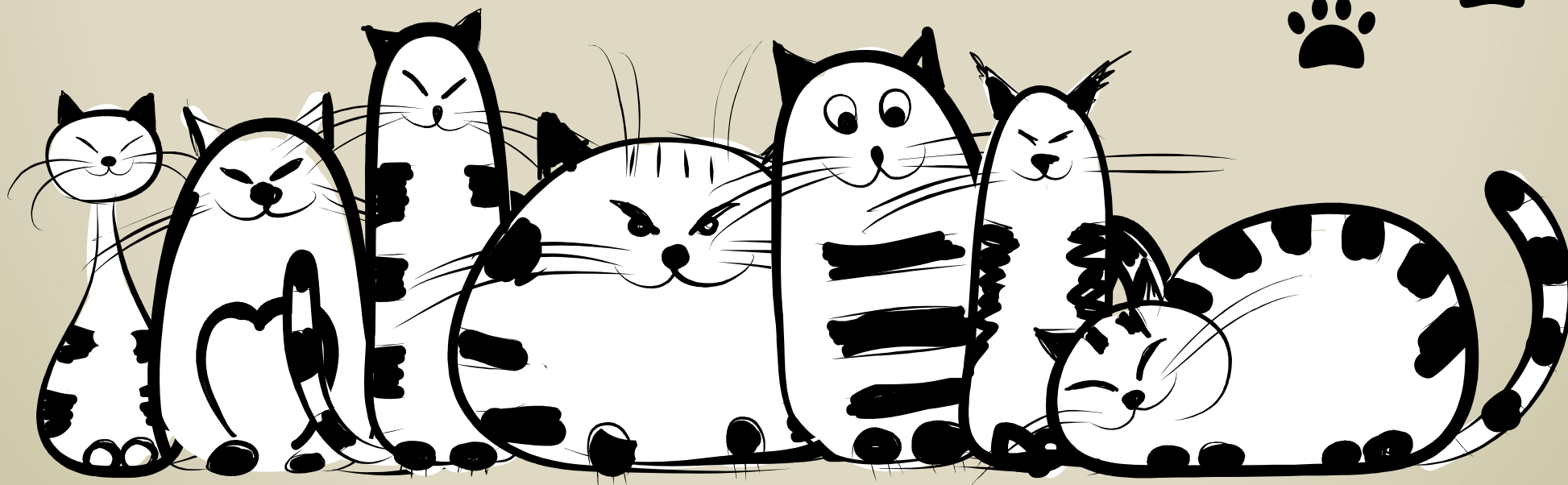
🐾 隐私审计开山之作 🐾

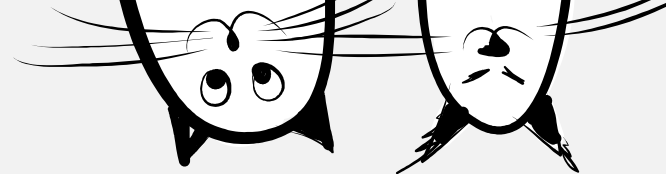
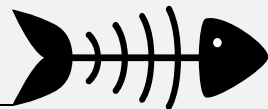
Adversary Instantiation: Lower Bounds for Differentially Private Machine Learning

(敌手实例化: DPML的下界)

出自: SP2021

作者: Milad Nasr 马萨诸塞大学阿默斯特分校、谷歌大脑





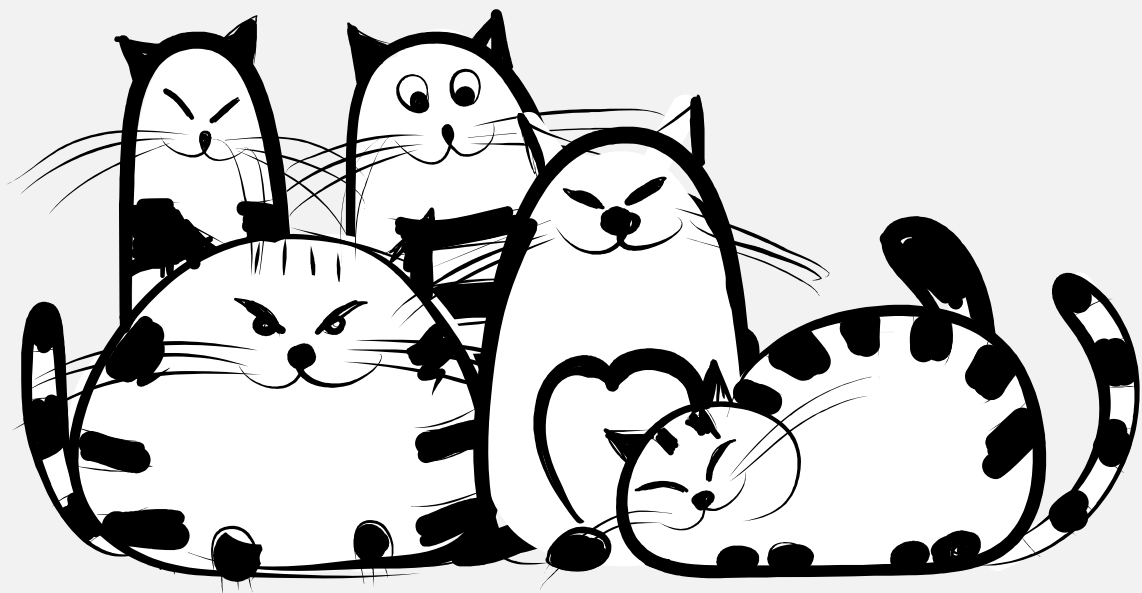
Section	Experiment	Access	Modification	Dataset	Protocol	Adversaries		Results
§ IV-A	API access	Black-box	Random Sample	Not modified	Protocol 1	Crafter 1	Distinguisher 1	Fig. 3
§ IV-B	Static Poison Input	Final Model	Malicious Sample	Not modified	Protocol 1	Crafter 2	Distinguisher 1	Fig. 4
§ IV-C	Intermediate Attack	Intermediate Models	Malicious Sample	Not modified	Protocol 2	Crafter 2	Distinguisher 2	Fig. 5
§ IV-D	Adaptive Poison Input	Intermediate Models	Adaptive Sample	Not modified	Protocol 3	Crafter 3	Distinguisher 2	Fig. 6
§ IV-E	Gradient Attack	Intermediate Models	Malicious Gradient	Not modified	Protocol 4	Crafter 4	Distinguisher 3	Fig. 7
§ IV-F	Dataset Attack	Intermediate Models	Malicious Gradient	Malicious	Protocol 4	Crafter 5	Distinguisher 3	Fig. 8

TABLE I: **Experiment settings.** We run attacks under six set of adversary capabilities; each experiment increases the capabilities of the adversary ranging from the weakest membership inference adversary to the strongest adversary with full DP capabilities.

由于计算限制, 我们只能执行 1,000 次攻击。这些实验花费了 3000 个 GPU 小时, 在 24 个 GPU 上并行化。

目录

CONTENTS



PART 01

Click here to add your text



PART 02

Click here to add your text



PART 03

Click here to add your text



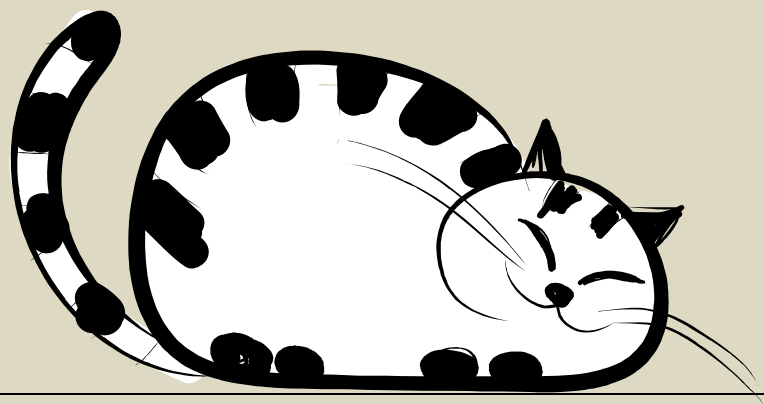
PART 04

Click here to add your text



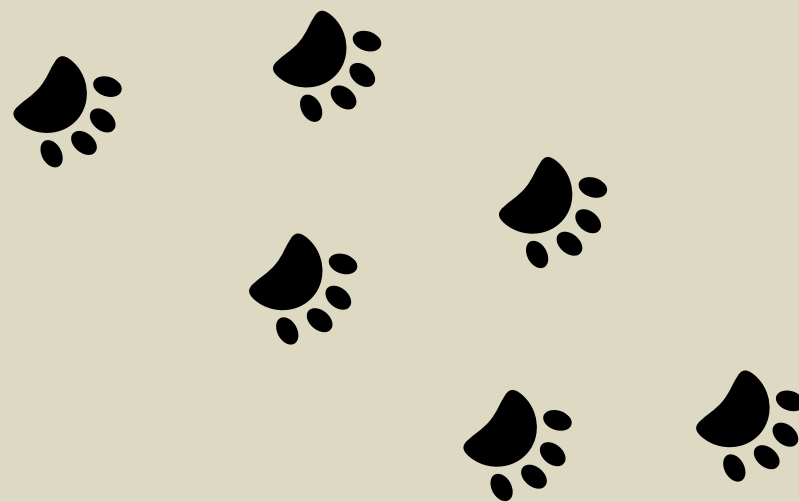
PART 05

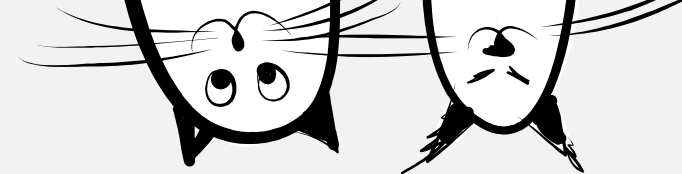
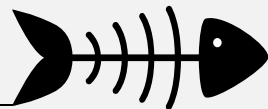
Click here to add your text



PART 01

相邻数据集构造器 Crafter(工匠)



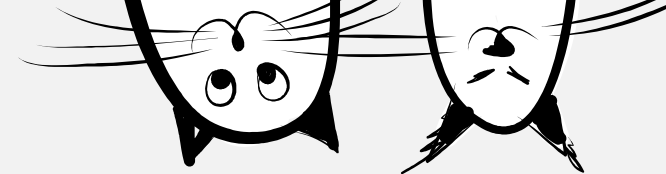
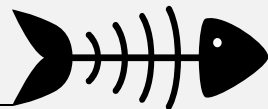


Crafter 1 Membership Inference Adversary

Require: existing training dataset D , underlying data distribution $\mathcal{D}$

- 1: $(x, y) \xleftarrow{\$} \mathcal{D}$
- 2: $D' \leftarrow D \cup \{(x, y)\}$
- 3: **return** D, D'

- D 是训练集， $\mathcal{D}$ 是 D 的分布
- 从分布中随机采样($\$$)一个数据拼接给 D ，得到相邻数据集 D'



Crafter 2 Static Adversary Crafting 静态对抗工匠

Require: train dataset D_{shadow} , adversarial train steps T , input learning rate s , model learning rate lr , training dataset D

1: $f_1^{shadow}, \dots, f_n^{shadow} \leftarrow \mathcal{T}_{dpsgd}(D_{shadow})$

2: $x, y \xleftarrow{\$}$ random sample from D_{shadow}

3: **for** T times **do**

4: $l_{malicious} \leftarrow \frac{1}{n} \sum_{i=1}^n \ell(f_i^{shadow}, x, y)$

5: $x \leftarrow x + s \nabla_x l_{malicious}$

6: **end for**

7: $D' = D \cup \{(x, y)\}$

8: **return** D, D'

Algorithm 3 Input canary generation in white-box setting

Args: In-distribution dataset D , model loss function l , model parameters

θ , T crafting steps, η step size

$\vec{g}_{dist} = \frac{1}{|D|} \sum_{(x_i, y_i) \in D} \nabla l(\theta, (x_i, y_i))$

$l_{adv}(x, y) = \left| \frac{\nabla l(\theta, (x, y)) \cdot \vec{g}_{dist}}{|\nabla l(\theta, (x, y))| |\vec{g}_{dist}|} \right|$

$(x, y) \xleftarrow{\$} D$

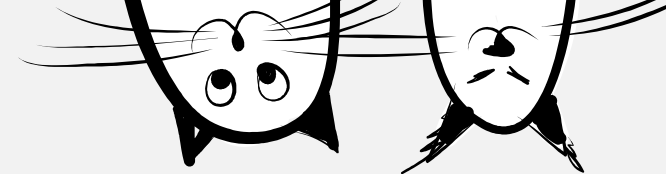
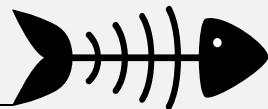
for $t \in \{T\}$ **do**

$x = x - \eta \nabla l_{adv}(x, y)$

end for

return (x, y)

- 这个工匠2有点[1]中算法3的意思，他是对数据进行了迭代，目的是增加影子模型对金丝雀数据(x,y)的判别loss。（增加loss是因为跟他相邻数据集构造及判别方式相关，降低loss也可以）
- 这样操作以后，影子模型更容易判别出这个迭代后的金丝雀(x,y)



Crafter 3 Dynamic Malicious Generation

Require: current model f_{θ_T} , train steps T , input learning rate s , model learning rate lr

1: $(D_a, D'_a) \leftarrow \text{Crafter 2}$

2: $x_{query}, y_{query} \leftarrow D'_a \setminus D_a$

3: $x, y \leftarrow \vec{0}$

4: **for** T times **do**

5: $l_{model} \leftarrow \ell(f, x, y)$

6: $f'(x) \leftarrow f(x) - lr \times \nabla_{f_{\theta_T}} l_{model}$

7: $l_{malicious} \leftarrow \ell(f', x_{query}, y_{query})$

8: $x \leftarrow x + s \nabla_x l_{malicious}$

9: **end for**

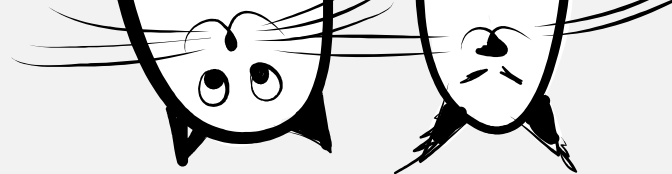
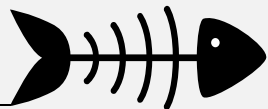
10: $D' = D \cup \{(x, y)\}$

11: **return** D, D'

} 模型f见到(x,y)就会给出一个小loss;

} (x,y)朝着f见到(x,y)就会对(x_q,y_q)给出大loss的方向迭代;

相当于这个(x,y)是(x_q,y_q)的一个后门，如果f学习了(x,y)，就会对(x_q,y_q)的loss非常大

**Crafter 4 Gradient attack**

Require: intermediate models up to step T $f^1, \dots, f^t$, clipping norm C , number of poison parameters $2n$, number of measurements T_o , training dataset D

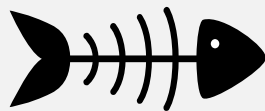
```
1:  $M \leftarrow \sum_{t=1}^{\min(T_o, T)-1} |f^{t+1} - f^t|$ 
2:  $points \leftarrow$  select smallest  $2n$  arguments from  $M$ 
3:  $\nabla_{malicious} \leftarrow \vec{0}$ 
4:  $s \leftarrow 1$ 
5: for  $p$  in  $points$  do
6:    $\nabla_{malicious}[p] = s \frac{C}{\sqrt{2n}}$ 
7:    $s = -s$ 
8: end for
9:
10:  $\mathcal{G} \leftarrow \{\}$ 
11: for  $(x, y) \in D$  do
12:    $\mathcal{G}.insert(\nabla l(f, x, y))$ 
13: end for
14: return  $\mathcal{G}, \mathcal{G} \cup \nabla_{malicious}$ 
```

构造了一个金丝雀梯度：这个梯度里的 $2n$ 个位置，要么变成 $C/\sqrt{2n}$ ，要么变成 $-C/\sqrt{2n}$

$2n$ 个位置是 T 轮梯度累和里的大梯度的最小 $2n$ 个位置

制作者将水印插入到模型参数中。为了最小化修改这些参数的影响，对抗者选择一组模型参数，这些参数对模型在训练数据上的性能影响较小。制作者修改梯度幅值最小的模型参数：直观地说，这些模型参数在训练过程中更新不多，因此它们是被修改的良好候选者。特别是对于训练的前几轮，对抗者观察模型参数的梯度，并在若干轮后，选择 $2n$ 个具有最小梯度绝对值总和的参数。制作者4概述了这一过程。

Crafter 5



Crafter 5 Malicious dataset

Require: initial model f_0 , dataset size n , input space $\mathcal{D}$, clipping norm C , selected params $2n$

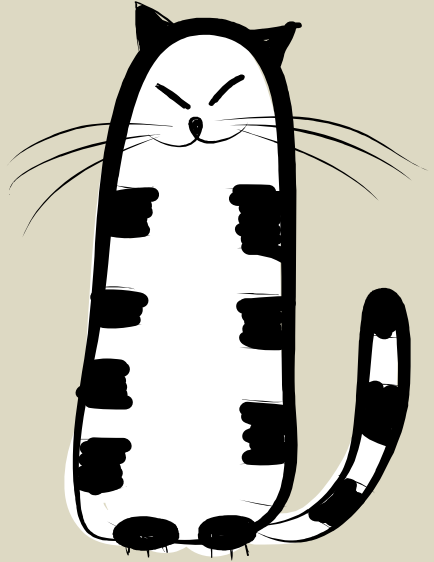
- 1: $D_{data} \leftarrow$ randomly sample n random instances from $\mathcal{D}$
- 2: $D_{label} \leftarrow f_0(D_{data})$ 用初始模型预测的label来当数据的label
- 3: $D \leftarrow (D_{data}, D_{label})$
- 4: **return** Call Crafter 4 on D

通过构造数据，让梯度变0；而不是传统的ML任务——优化模型让梯度变0；甚至直接把后续的训练禁用了（学习率置为0）但是如果是在梯度上裁剪加噪的话，那DP保护也没达到效果？

a) 制造者：（Crafter 5）为了实现这一点，我们将设计数据集 D ，以便如果 $B = \mathbb{B}(D') \subset D$ ，那么在这个小批量 $S(B)$ 上训练一步将最小程度地扰动神经网络的权重。我们设计Crafter 5来创建恶意数据集。非正式地说，该算法的步骤如下。使用神经网络的初始随机参数 f_{θ_0} （假设在DP-SGD分析中首次被对手调用时会被对手知晓），我们构建一个数据集 D ，模型已经能够完美地标记。因为标记是完美的，我们将有

$$\nabla_{f_{\theta}}(\mathcal{L}(f_{\theta_0}, D)) \equiv 0 \quad \forall D \subset D$$

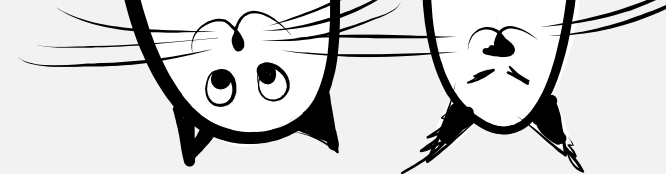
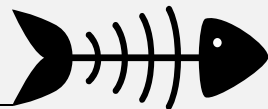
（否则可以构建一个更好的数据集，导致矛盾），因此，在不包含 $D' \setminus D$ 的任何小批量上训练将是一个有效的无操作——除了通过高斯机制添加的噪声。不幸的是，模型添加的这种高斯噪声将破坏模型权重，并导致未来的梯度更新不为零。为了防止这种情况，我们将训练算法中的学习率设置为零——基本上忽略了实际更新，因此 $\theta_i \equiv \theta_0$ 。这是允许的，因为DP-SGD保证了任何超参数的分配都将导致一个私有模型——即使超参数的选择是病理性的，并且永远不会被现实对手使用。



PART 02

敌手(审计者) Distinguisher



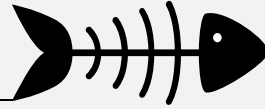


Distinguisher 1 Membership Inference Adversary

Require: model f_θ , query input (x, y) , threshold τ

```
1:  $L \leftarrow \ell(f_\theta, x, y)$   
2: if  $L \leq \tau$  then  
3:   return  $D'$   
4: end if  
5: return  $D$ 
```

- 计算 (x,y) 的loss，如果小于阈值，就表示吃过这个数据，来自于 D' ；否则没吃过，来自于 D
- D' 是由 D 删减一个得到的



Distinguisher 2 White-box Membership attack

Require: all intermediate steps to step T $f_1, \dots, f_T$, query input (x, y) , threshold τ , attack method (max or mean)

- 1: **if** attack method is max **then**
- 2: $L \leftarrow \max_{i=1}^T \ell(f_{\theta_i}, x, y)$
- 3: **end if**
- 4: **if** attack method is mean **then**
- 5: $L \leftarrow \frac{1}{T} \sum_{i=1}^T \ell(f_{\theta_i}, x, y)$
- 6: **end if**
- 7: **if** $L < \tau$ **then**
- 8: **return** D'
- 9: **end if**
- 10: **return** D

白盒也就是，能访问 $i=1\dots T$ 中的每一轮的模型

Distinguisher 3

高斯分布概率密度函数:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

Distinguisher 3 Watermark detection

Require: subsampling rate q , number of iterations T , clipping norm C , number of poison parameters $2n$, number of measurements t_o , trained models $f_0, \dots, f_T$, threshold τ

```
1:  $M \leftarrow \vec{0}$ 
2:  $p_{\text{watermark}} \leftarrow 1$ 
3: for  $t = 1$  to  $T$  do
4:   if  $t \leq t_o$  then           先训练 $t_o$ 轮，用来找
5:      $M = M + |f_t - f_{t-1}|$       梯度最小的 $2n$ 个点
6:   else
7:      $\text{points} \leftarrow$  select smallest  $2n$  arguments from  $M$ 
8:      $\nabla_{\text{malicious}} \leftarrow \vec{0}$ 
9:      $s \leftarrow 1$ 
10:     $d \leftarrow 0$ 
11:    for  $p$  in  $\text{points}$  do         这里是不是没写清楚，
12:       $d = d + s \cdot f_t - f_{t-1}[p]$    $s \cdot [f_t[p] - f_{t-1}[p]]?$ 
13:       $s = -s$ 
14:    end for
15:     $p_{\text{watermark}} = p_{\text{watermark}} \times p(d|\text{watermark})$ 
16:  end if           如果 $p_{\text{watermark}} \rightarrow 0$ ，那越乘越小；反之越
17: end for           乘越接近1； $\tau$ 的选择就比较宽松了
18: if  $p_{\text{watermark}} \geq \tau$  then
19:   return  $D'$ 
20: end if
21: return  $D$ 
```

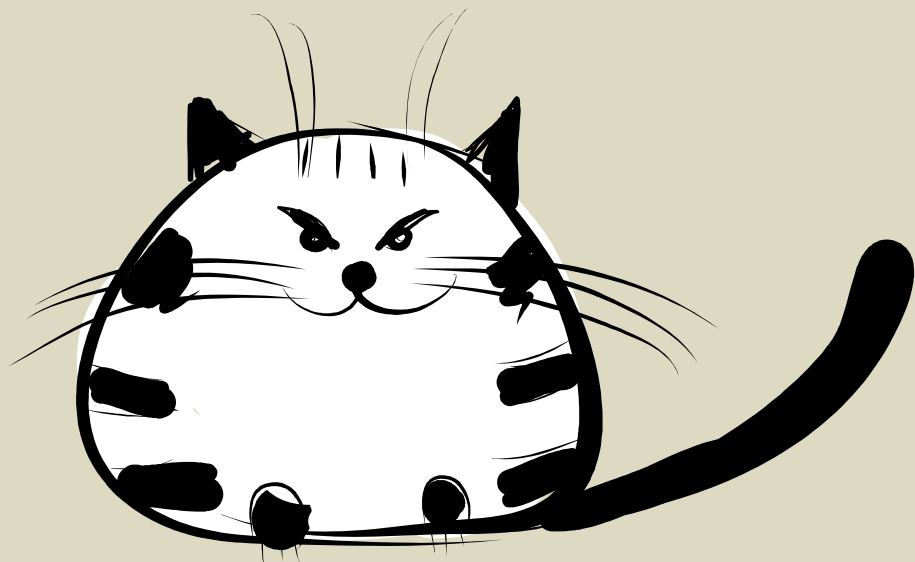
给定一组模型权重 $\{\theta_i\}_{i=1}^N$ ，区分器对抗者将直接检查模型权重以进行猜测。与制作者类似，对抗者观察前 t_o 轮的梯度，找出哪 $2n$ 个模型参数具有最小的总体绝对值。由于制作者试图增加这些模型参数之间的距离，为了检测模型参数中是否存在水印，区分器计算这些参数之间的距离，然后进行假设检验以检测水印的存在。参数集的距离要么来自梯度分布（ Z ）加上添加的高斯噪声（即，零假设），要么是水印参数，即它们来自梯度分布加上添加的高斯噪声再加上水印：

$$H_{\text{null}} : Z + \mathcal{N}(0, \sigma^2) \quad H_{\text{watermark}} : Z + \mathcal{N}\left(\frac{C}{\sqrt{2n}}, \sigma^2\right)$$

由于制作者选择的是变化最小的参数，我们假设这些点的梯度为零。这使得我们可以将问题简化为判断这些参数之间的距离是来自均值为零的高斯分布，还是来自均值为水印参数之间距离之和的高斯分布。现在，给定水印参数之间的距离，我们可以计算它来自水印梯度的可能性：

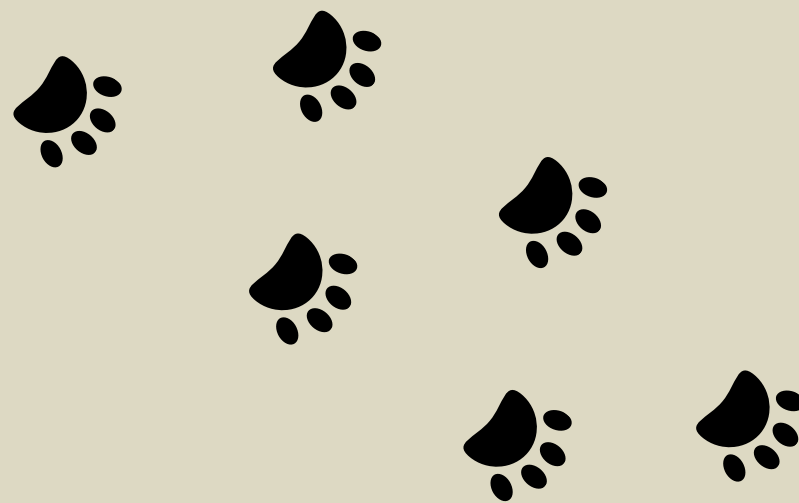
$$p(\text{Distance} = d \mid \text{watermark}) = \frac{1}{2\sqrt{\pi n \sigma^2}} \exp - \left(\frac{(x - C)^2}{2\sigma^2} \right)$$

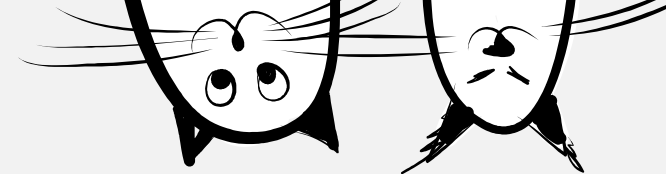
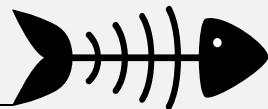
其中， $2n$ 是选择的参数数量， C 和 σ 是裁剪范数和用于裁剪与噪声梯度的规模，在DP-SGD中使用， d 是区分器3计算的距离。



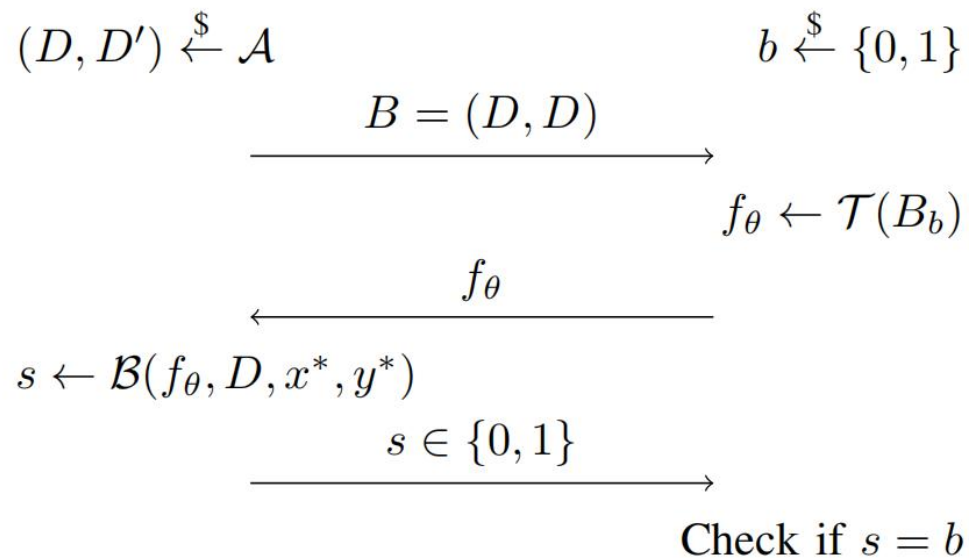
PART 03

协议 Protocol





Membership Inference Adversary Game	Adversary
Model Trainer	



针对成员推断攻击的对抗游戏。

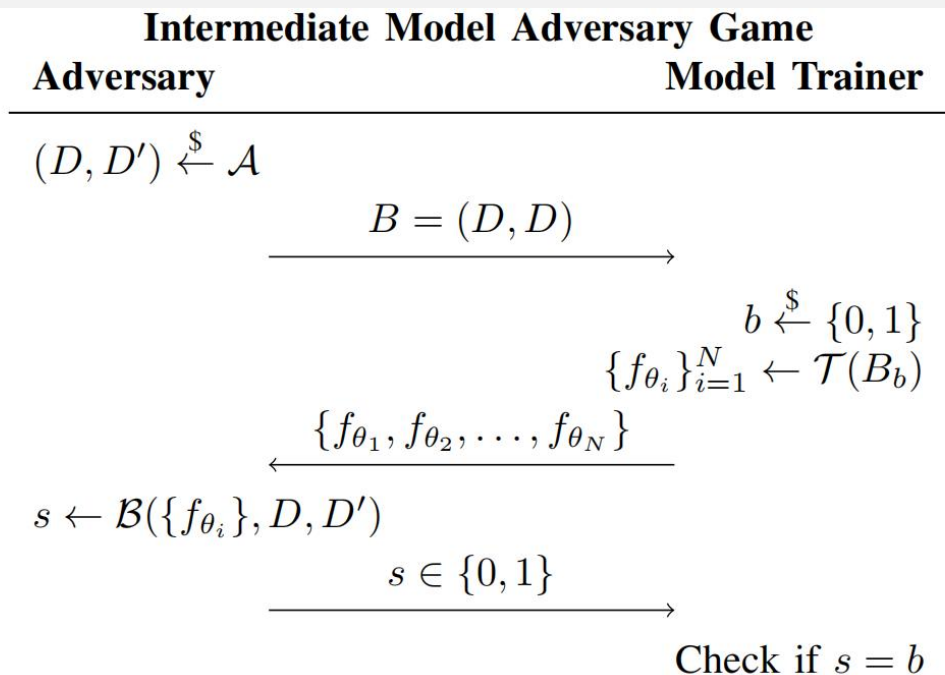
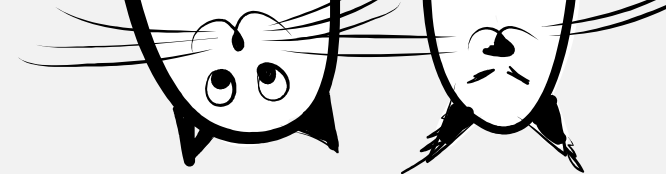
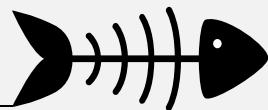
第一轮： 对抗者选择两个数据集。

第二轮： 模型训练者在其中一个数据集上随机训练一个模型，并返回该模型。

第三轮： 对抗者预测用于训练的具体数据集。

Protocol 1: Adversary game for membership inference attack. Round 1. The adversary chooses two datasets. **Round 2.** The model trainer randomly trains a model on one of these, and returns the model. **Round 3.** The adversary predicts which dataset was used for training.

Protocol 2



针对中间模型对手的对抗游戏。

第一轮： 对手选择两个数据集。

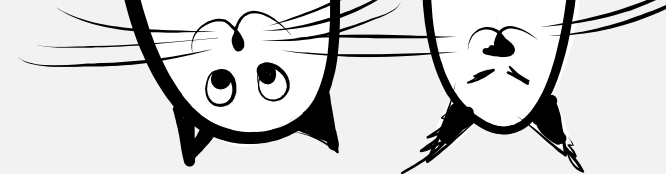
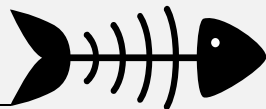
第二轮： 模型训练者随机选择其中一个数据集进行训练，并返回完整的模型更新序列。

第三轮： 对手预测哪一个数据集被用于训练。

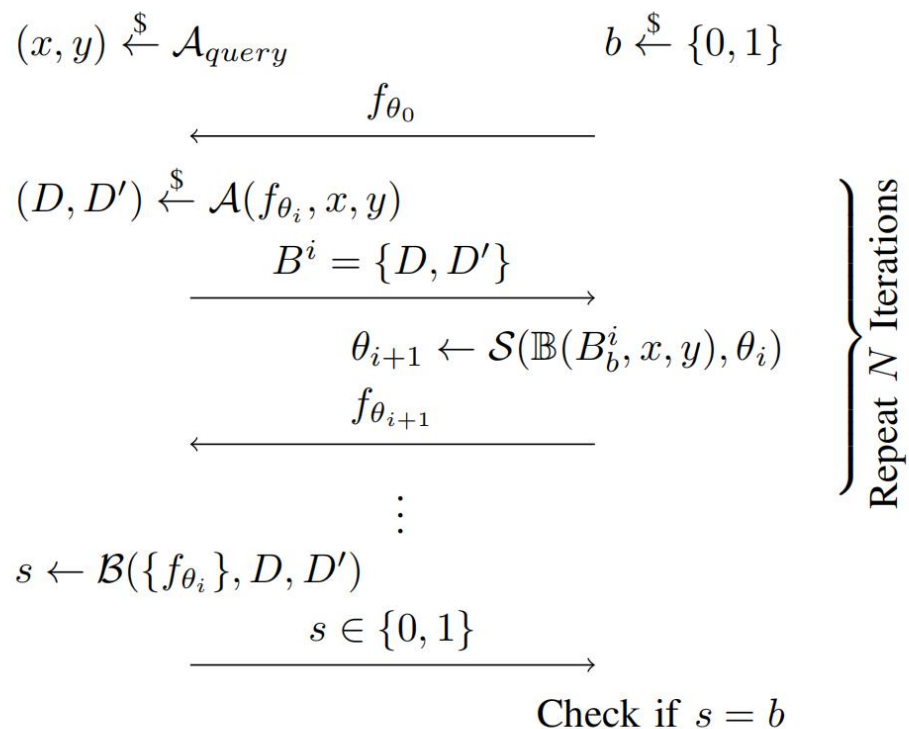
主要区别就是，训练的**N**轮的所有模型都被返回了；敌手可以访问每一轮的模型

Protocol 2: Adversary game for intermediate model adversary. Round 1. The adversary chooses two datasets. **Round 2.** The model trainer randomly trains a model on one of these, and returns *the full sequence of model updates*. **Round 3.** The adversary predicts which dataset was used for training.

Protocol 3



Adaptive Input Poisoning Adversary Game



适应性输入中毒攻击的对抗性游戏。

第0回合。 对抗者生成查询输入。

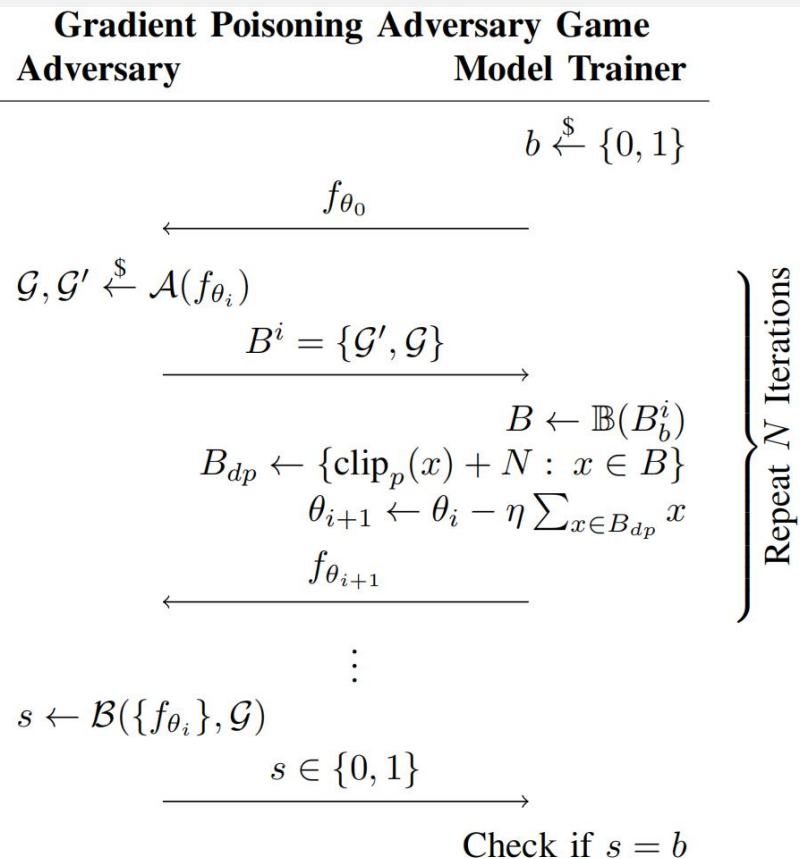
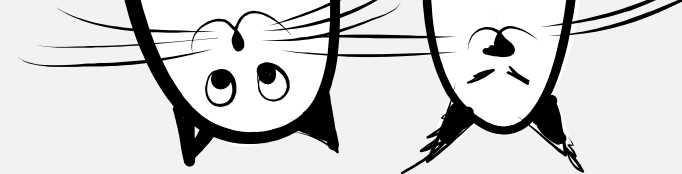
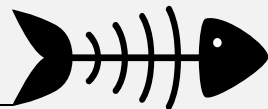
第2i回合。 对抗者根据当前的权重 θ_i 选择数据集 D 、 D' 。

第2i+1回合。 模型训练者在这些数据集中的一个小批量的训练。

第N回合。 对抗者预测用于训练的数据集是哪一个。

Protocol 3: Adversary game for adaptive input poisoning attack. **Round 0.** The adversary generates a *query input*. **Round 2i.** The *adversary* chooses datasets D, D' given the current weights θ_i . **Round 2i+1.** The *model trainer* trains for one minibatch on one of these datasets. **Round N.** The adversary predicts which dataset was used for training.

Protocol 4



梯度中毒攻击的对抗性游戏。

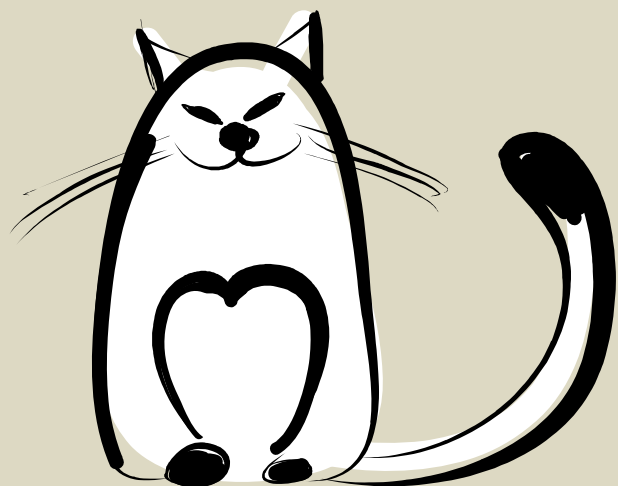
第2i回合。对抗者选择一组梯度 $\mathcal{G}$ 、 $\mathcal{G}'$ 。

第2i+1回合。模型训练者选择这些梯度更新中的一个子集，并使用被裁剪和加噪声的梯度更新参数。

第2N回合。对抗者预测用于训练的数据集是哪一个。

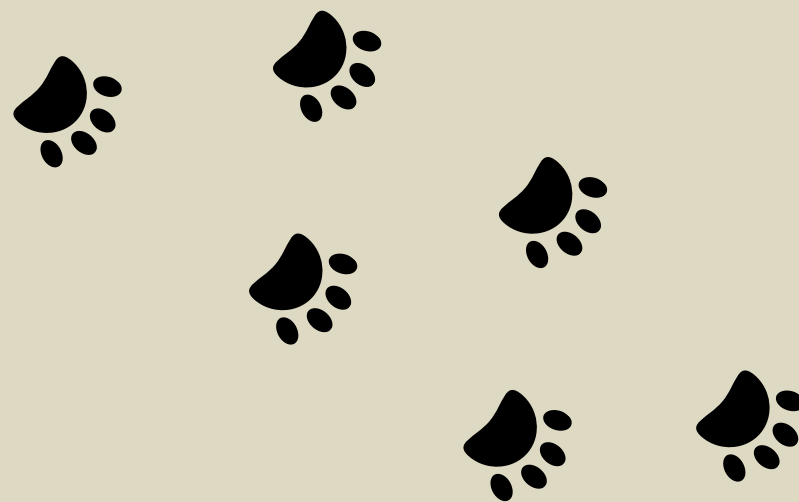
Protocol 4: **Adversary game for gradient poisoning attack.**

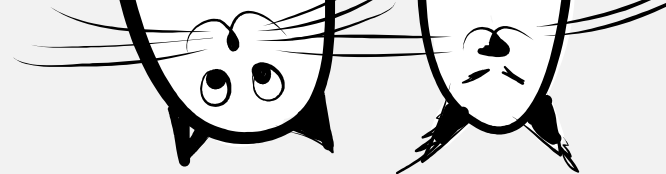
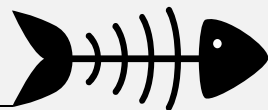
Round 2i. The adversary chooses a collection of gradients $\mathcal{G}, \mathcal{G}'$. **Round 2i+1.** The model trainer chooses a subset of one of these gradient updates, and updates the parameters using the clipped and noised gradients. **Round 2N.** The adversary predicts which dataset was used for training.



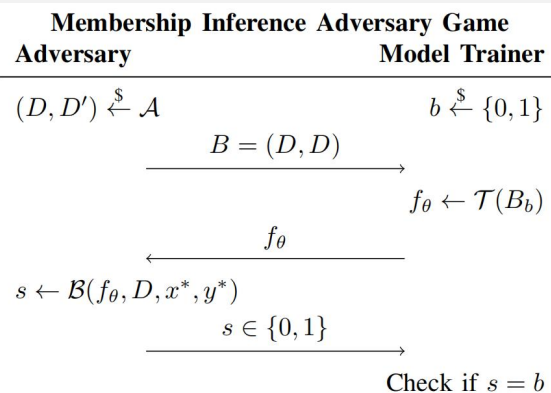
PART 04

Experiment 实验





黑盒访问、随机采样、不改动数据集、协议1、工匠1、敌手1



Protocol 1: Adversary game for membership inference attack. Round 1. The adversary chooses two datasets. **Round 2.** The model trainer randomly trains a model on one of these, and returns the model. **Round 3.** The adversary predicts which dataset was used for training.

Crafter 1 Membership Inference Adversary

Require: existing training dataset D , underlying data distribution $\mathcal{D}$

- 1: $(x, y) \xleftarrow{\$} \mathcal{D}$
- 2: $D' \leftarrow D \cup \{(x, y)\}$
- 3: **return** D, D'

Distinguisher 1 Membership Inference Adversary

Require: model f_θ , query input (x, y) , threshold τ

- 1: $L \leftarrow \ell(f_\theta, x, y)$
- 2: **if** $L \leq \tau$ **then**
- 3: **return** D'
- 4: **end if**
- 5: **return** D

其实就是训练过程完全打包了，只开放了输入和输出API给敌手

当我们在 CIFAR-10 数据集上执行该攻击，并使用 $\epsilon = 4$ 的差分隐私训练模型时，该攻击的真阳率为 0.017，而假阳率为 0.002。利用 Clopper-Pearson 方法，我们可以以概率方式为该攻击性能构造下界；例如，在进行了 1000 次试验后，有 95% 的概率该攻击的假阳性率低于 0.01。利用公式 (4) 和 (5)，我们可以将 (ϵ, δ) -差分隐私的经验下界转换为 $(0.31, 10^{-5})$ 。这一数值明显低于经证明正确的上界 $(4, 10^{-5})$ 。

$$\epsilon_{\text{empirical}} = \max\left(\log \frac{1 - \delta - FP}{FN}, \log \frac{1 - \delta - FN}{FP}\right) \quad (4)$$

$$\epsilon_{\text{empirical}}^{\text{lower}} = \max\left(\log \frac{1 - \delta - FP^{\text{high}}}{FN^{\text{high}}}, \log \frac{1 - \delta - FN^{\text{high}}}{FP^{\text{high}}}\right) \quad (5)$$

这个例子写得非常简单，但是计算起来其实非常复杂，后面用两页PPT去讲解怎么计算的

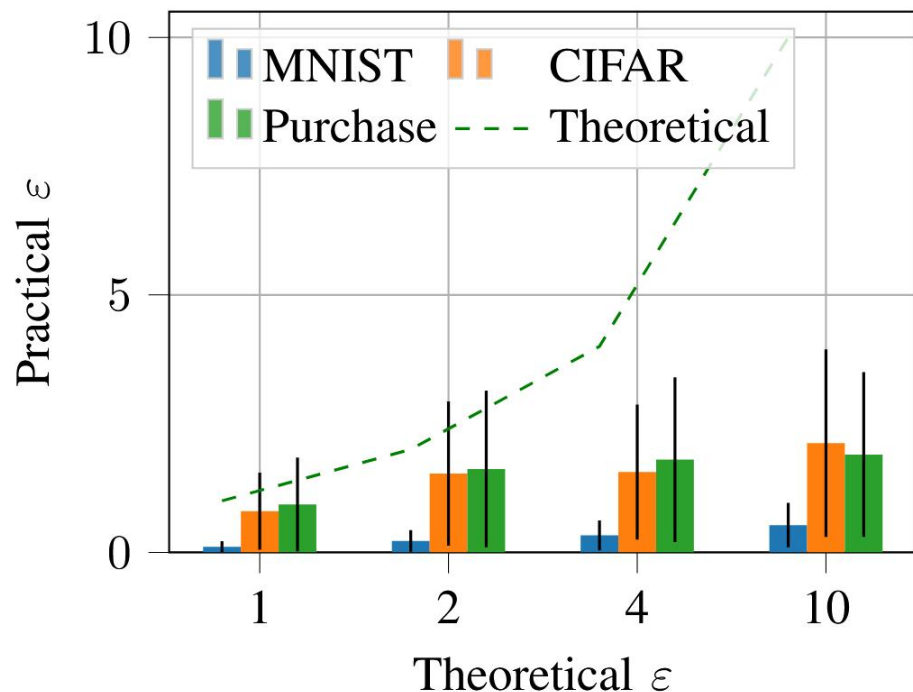
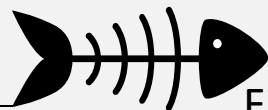


Fig. 3: **Membership inference attack:** the adversary only adds one sample from the underlying data distribution.

Figure 3 展示了其他数据集和不同差分隐私值的经验 ϵ 值。对于MNIST数据集，对手的优势并没有显著超过随机机会；我们假设这是因为MNIST图像都非常相似，因此向数据集中插入或删除任何一个训练样本并不会对模型造成太大改变。然而，CIFAR-10和Purchase数据集的任务多样性(much more diverse tasks)更大，对手可以更容易地区分两个数据集 D 和 D' 。总体而言，API访问攻击并未利用DP-SGD分析中假定对手可用的任何假设，因此从底层数据分布中得到的平均输入并未泄漏像理论上界所建议的那么多私密信息。然而，随着任务变得更加复杂，信息泄漏增加。对于这一结果，有两种可能的解释，这将是本文的主要焦点：

- 1) 解释1：差分隐私的上界过于悲观。DP-SGD提供的隐私比可以证明的上界强得多。
- 2) 解释2：这种攻击比较弱。一个更强的攻击可能会更经常成功，因此DP-SGD提供的界限可能是准确的。先前的工作经常观察到这种现象，例如使用 $\epsilon = 10^5$ -DP [6]来训练模型，尽管从理论上几乎没有提供隐私，因为从经验上看隐私效果强大。然而，正如我们在介绍中已经揭示的那样，结果表明解释2是正确的：尽管对于这种弱攻击来说，界限较松，但这并不意味着当对手利用所有能力时，DP-SGD本身是松散的。

Experiment 2 Static Input Poisoning Adversary



最终模型访问、恶意采样、不改动数据集、协议1(MIA)、工匠2(加强对手能力)、敌手1(MIA, 按loss阈值判断)

```
Crafter 2 Static Adversary Crafting

Require: train dataset  $D_{shadow}$ , adversarial train steps  $T$ , input
learning rate  $s$ , model learning rate  $lr$ , training dataset  $D$ 
1:  $f_1^{shadow}, \dots, f_n^{shadow} \leftarrow \mathcal{T}_{dpsgd}(D_{shadow})$ 
2:  $x, y \xleftarrow{\$}$  random sample from  $D_{shadow}$ 
3: for  $T$  times do
4:    $l_{malicious} \leftarrow \frac{1}{n} \sum_{i=1}^n \ell(f_i^{shadow}, x, y)$ 
5:    $x \leftarrow x + s \nabla_x l_{malicious}$ 
6: end for
7:  $D' = D \cup \{(x, y)\}$ 
8: return  $D, D'$ 
```

如图4所示，该对手能够利用其数据中毒能力泄露更多私密信息，并相较于之前的攻击获得更高的经验下界。这一现象在所有数据集中均保持一致。原因在于，经过中毒的输入是针对成员推断的最坏情况输入，而之前的攻击是在训练分布中抽取的平均情况输入上进行的。结果表明，DP-SGD给出的隐私界限至少低估了10倍的风险。[最后一句在说啥看不懂](#)

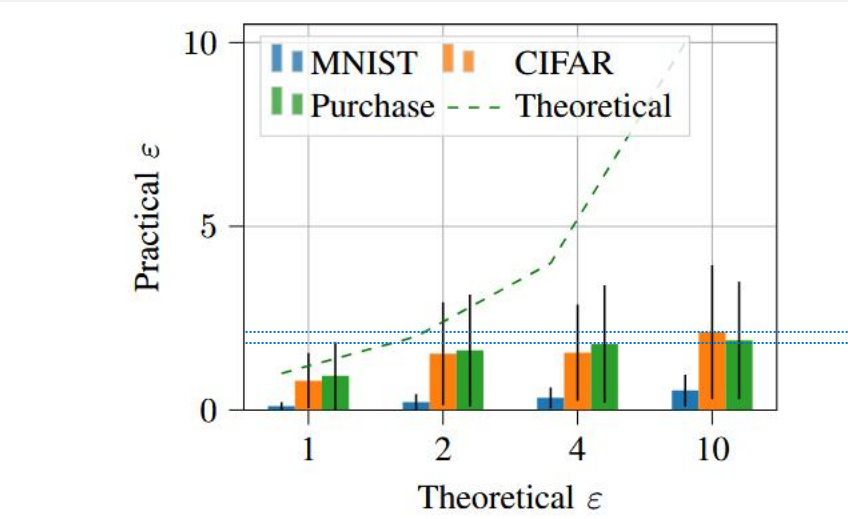


Fig. 3: **Membership inference attack:** the adversary only adds one sample from the underlying data distribution.

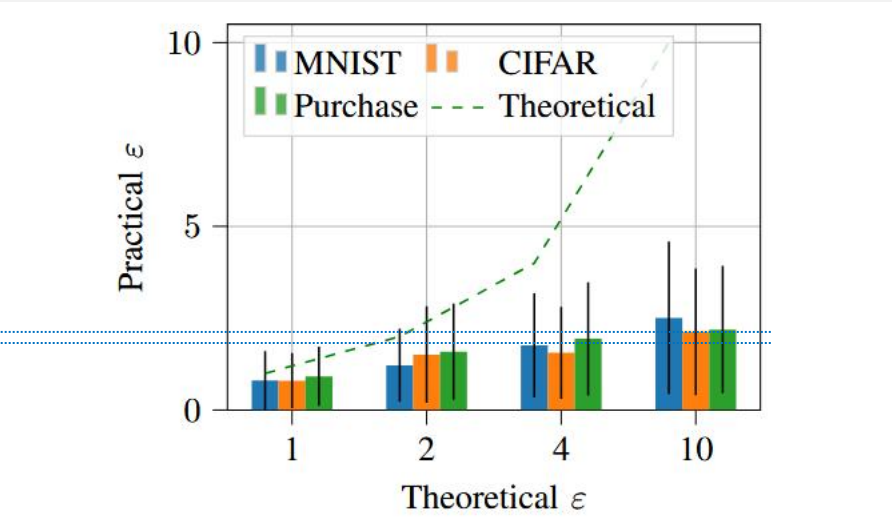
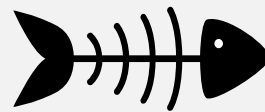


Fig. 4: **Malicious input attack:** the adversary has blackbox access. The maliciously crafted input leaks more information than a random sample from the data distribution.

在MNIST上好很多，但是另外两个数据集没有好特别多

Experiment 3 Intermediate Poison Attack



中间模型访问、恶意采样、不改动数据集、协议2(中间模型敌手)、工匠2、敌手2(白盒成员推理攻击)

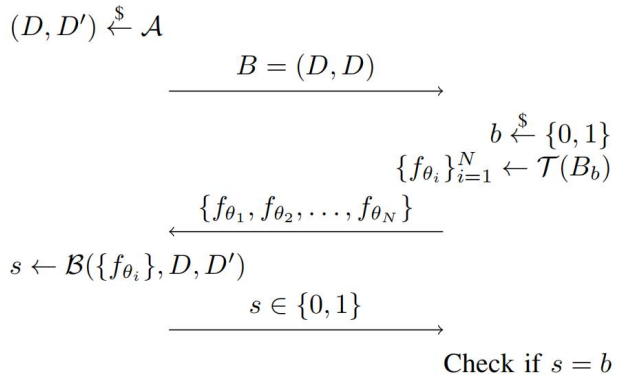
Crafter 2 Static Adversary Crafting

Require: train dataset D_{shadow} , adversarial train steps T , input learning rate s , model learning rate lr , training dataset D

```
1:  $f_1^{shadow}, \dots, f_n^{shadow} \leftarrow \mathcal{T}_{dpsgd}(D_{shadow})$ 
2:  $x, y \xleftarrow{\$}$  random sample from  $D_{shadow}$ 
3: for  $T$  times do
4:    $l_{malicious} \leftarrow \frac{1}{n} \sum_{i=1}^n \ell(f_i^{shadow}, x, y)$ 
5:    $x \leftarrow x + s \nabla_x l_{malicious}$ 
6: end for
7:  $D' = D \cup \{(x, y)\}$ 
8: return  $D, D'$ 
```

Intermediate Model Adversary Game

Adversary **Model Trainer**



Protocol 2: **Adversary game for intermediate model adversary. Round 1.** The adversary chooses two datasets. **Round 2.** The model trainer randomly trains a model on one of these, and returns *the full sequence of model updates*. **Round 3.** The adversary predicts which dataset was used for training.

Distinguisher 2 White-box Membership attack

Require: all intermediate steps to step T $f_1, \dots, f_T$, query input (x, y) , threshold τ , attack method (max or mean)

```
1: if attack method is max then
2:    $L \leftarrow \max_{i=1}^T \ell(f_{\theta_i}, x, y)$ 
3: end if
4: if attack method is mean then
5:    $L \leftarrow \frac{1}{T} \sum_{i=1}^T \ell(f_{\theta_i}, x, y)$ 
6: end if
7: if  $L < \tau$  then
8:   return  $D'$ 
9: end if
10: return  $D$ 
```

(区分器2) 给定训练模型参数的序列，我们现在构建我们的第二个对手，以猜测训练者选择了哪个数据集。我们修改了“工匠1”，以便利用对训练中间输出的访问权限。与仅查看最终模型在中毒样本上的损失不同，我们的对手还分析了中间损失。我们计算所有中间步骤中**中毒样本的平均损失或最大损失**，并在损失小于某个阈值时猜测模型是在数据集D上训练的。我们在最大值和平均值变体之间选择了最佳攻击者。在我们的实验中，对于较小的epsilon ($\epsilon \leq 2$)，最大值的效果更好；而对于大于2的epsilon，平均值则更有效。区分器2概述了最终的攻击方法。

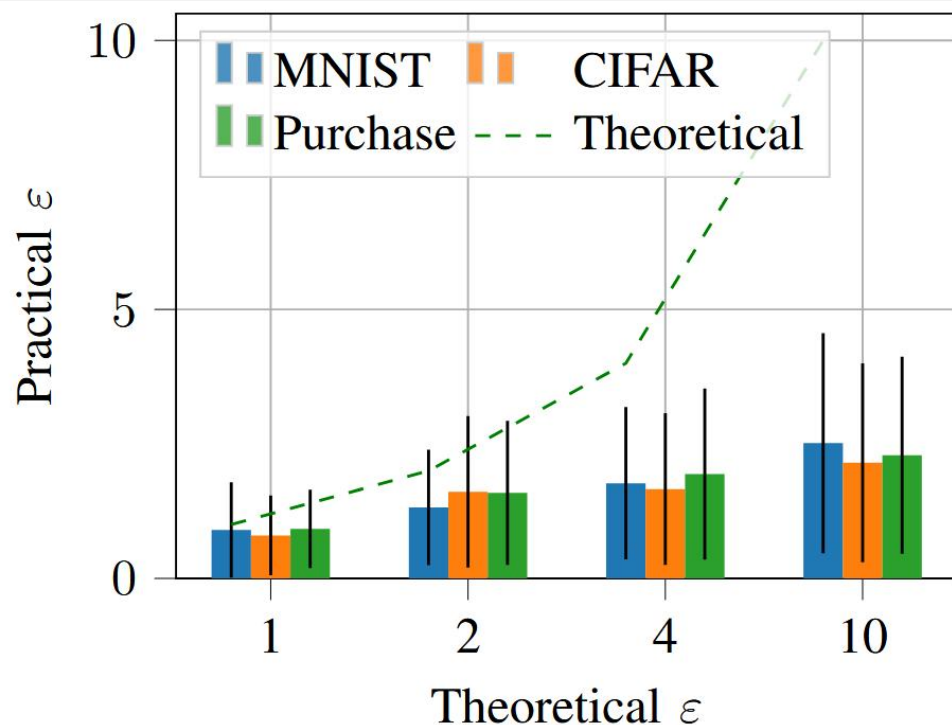
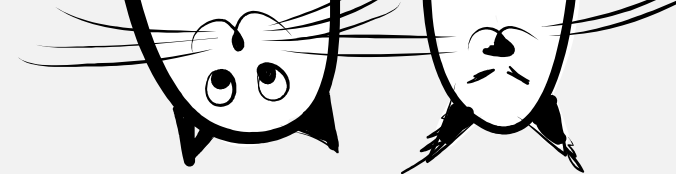
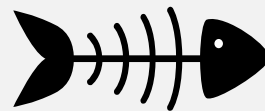
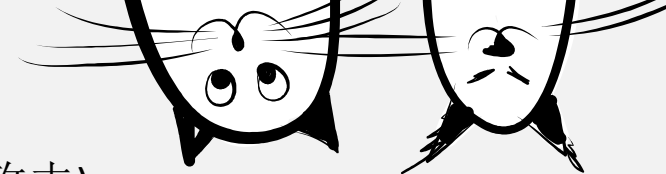
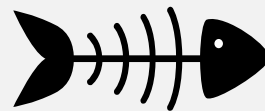


Fig. 5: **Malicious input attack:** the adversary has white-box access to the training dataset. The results are slightly better compared to the malicious input with blackbox access.

图5中的结果显示，这个对手仅略微优于我们之前的对手，而之前的对手仅能访问最终模型参数。这表明，训练算法输出的最终模型几乎泄露了与训练过程中应用的梯度同样多的信息。这一结果与凸模型的理论相符，尽管深度神经网络是非凸的。

Experiment 4 Adaptive Poisoning Attack



中间模型访问、自适应采样、不改动数据集、协议3、工匠3、敌手2(白盒成员推理攻击)

Distinguisher 2 White-box Membership attack

Require: all intermediate steps to step T $f_1, \dots, f_T$, query input (x, y) , threshold τ , attack method (max or mean)

```

1: if attack method is max then
2:    $L \leftarrow \max_{i=1}^T \ell(f_{\theta_i}, x, y)$  (x,y)→(x_query,y_query)
3: end if
4: if attack method is mean then
5:    $L \leftarrow \frac{1}{T} \sum_{i=1}^T \ell(f_{\theta_i}, x, y)$ 
6: end if
7: if  $L < \tau$  then
8:   return  $D'$ 
9: end if
10: return  $D$ 

```

Crafter 3 Dynamic Malicious Generation

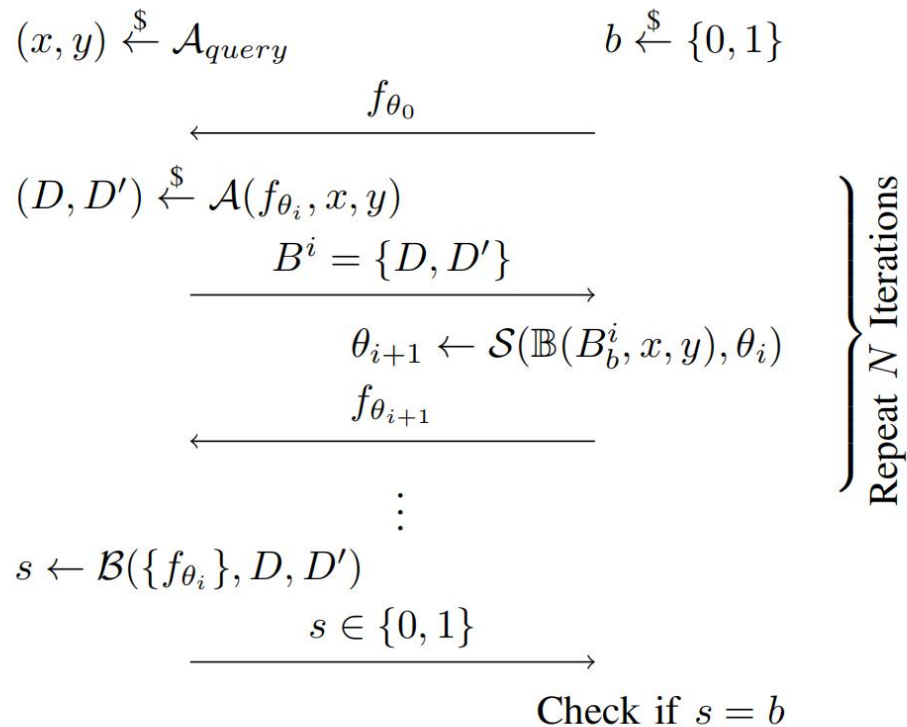
Require: current model f_{θ_T} , train steps T , input learning rate s , model learning rate lr

```

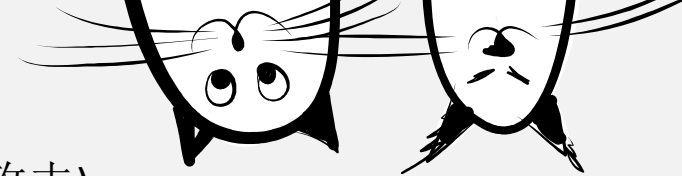
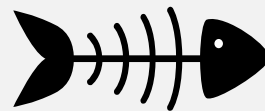
1:  $(D_a, D'_a) \leftarrow$  Crafter 2
2:  $x_{query}, y_{query} \leftarrow D'_a \setminus D_a$ 
3:  $x, y \leftarrow \vec{0}$ 
4: for  $T$  times do
5:    $l_{model} \leftarrow \ell(f, x, y)$ 
6:    $f'(x) \leftarrow f(x) - lr \times \nabla_{f_{\theta_T}} l_{model}$ 
7:    $l_{malicious} \leftarrow \ell(f', x_{query}, y_{query})$ 
8:    $x \leftarrow x + s \nabla_x l_{malicious}$ 
9: end for
10:  $D' = D \cup \{(x, y)\}$ 
11: return  $D, D'$ 

```

Adaptive Input Poisoning Adversary Game



Protocol 3: Adversary game for adaptive input poisoning attack. **Round 0.** The adversary generates a *query input*. **Round 2i.** The *adversary* chooses datasets D, D' given the current weights θ_i . **Round 2i+1.** The *model trainer* trains for one minibatch on one of these datasets. **Round N.** The adversary predicts which dataset was used for training.



中间模型访问、自适应采样、不改动数据集、协议3、工匠3、敌手2(白盒成员推理攻击)

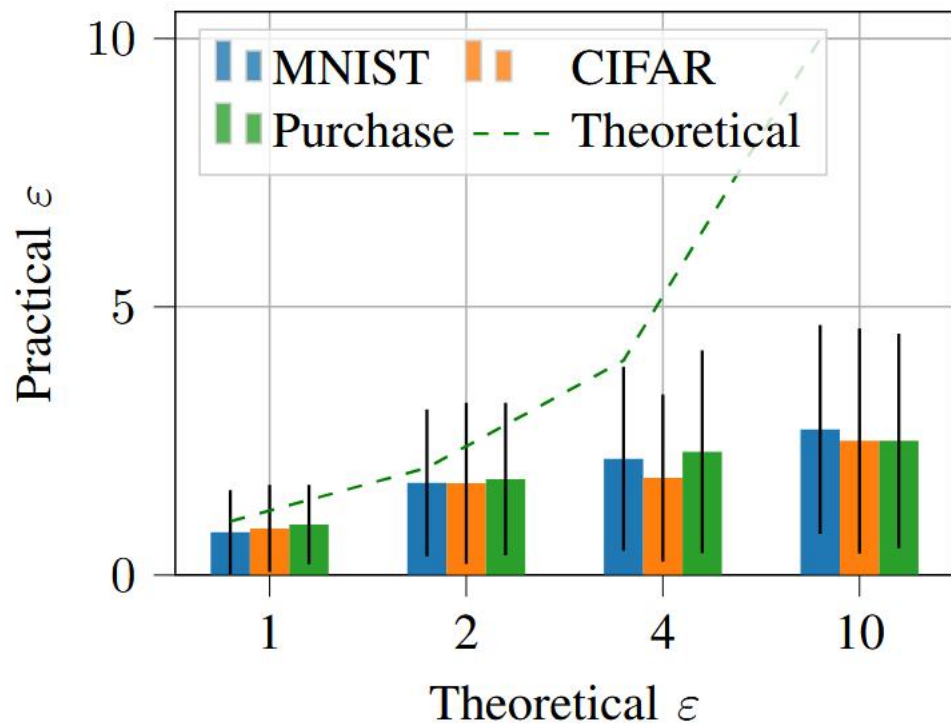
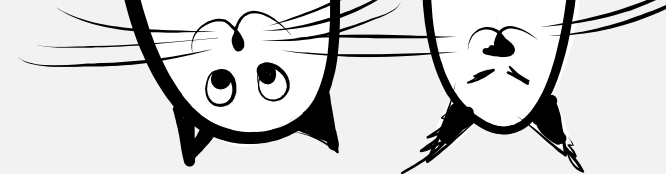
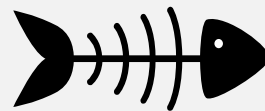


Fig. 6: **Adaptive malicious input attack:** the adversary changes the training dataset in each iteration. The adversary can achieve better results compared to prior attacks.

图 6 总结了此设置的结果。与之前的攻击相比，攻击者现在获得了更严格的隐私下限。例如，在 $\epsilon = 2$ 的 Purchase 数据集上，这种自适应中毒攻击可以实现下限 $\epsilon_{\text{lower}} = 0.37$ ，而上一节中为 0.25。能够在每次迭代时修改训练数据集是有效利用对中间模型更新的访问并获得更严格界限的缺失关键。



中间模型访问、恶意采样、不改动数据集、协议4、工匠4(梯度水印)、敌手3

Crafter 4 Gradient attack

Require: intermediate models up to step T $f^1, \dots, f^t$, clipping norm C , number of poison parameters $2n$, number of measurements T_o , training dataset D

```

1:  $M \leftarrow \sum_{t=1}^{\min(T_o, T)-1} |f^{t+1} - f^t|$ 
2:  $points \leftarrow$  select smallest  $2n$  arguments from  $M$ 
3:  $\nabla_{malicious} \leftarrow \vec{0}$ 
4:  $s \leftarrow 1$ 
5: for  $p$  in  $points$  do
6:    $\nabla_{malicious}[p] = s \frac{C}{\sqrt{2n}}$ 
7:    $s = -s$ 
8: end for
9:
10:  $\mathcal{G} \leftarrow \{\}$ 
11: for  $(x, y) \in D$  do
12:    $\mathcal{G}.insert(\nabla l(f, x, y))$ 
13: end for
14: return  $\mathcal{G}, \mathcal{G} \cup \nabla_{malicious}$ 

```

Distinguisher 3 Watermark detection

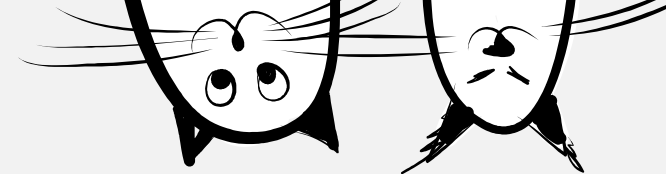
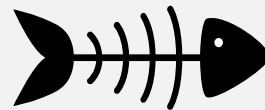
Require: subsampling rate q , number of iterations T , clipping norm C , number of poison parameters $2n$, number of measurements t_o , trained models $f_0, \dots, f_T$, threshold τ

```

1:  $M \leftarrow \vec{0}$ 
2:  $p_{watermark} \leftarrow 1$ 
3: for  $t = 1$  to  $T$  do
4:   if  $t \leq t_o$  then                                先训练 $t_o$ 轮，用来找
5:      $M = M + |f_t - f_{t-1}|$                             梯度最小的 $2n$ 个点
6:   else
7:      $points \leftarrow$  select smallest  $2n$  arguments from  $M$ 
8:      $\nabla_{malicious} \leftarrow \vec{0}$ 
9:      $s \leftarrow 1$ 
10:     $d \leftarrow 0$ 
11:    for  $p$  in  $points$  do                                这里是不是没写清楚，
12:       $d = d + s \cdot f_t - f_{t-1}[p]$                      $s*[f_t[p]-f_{t-1}[p]]?$ 
13:       $s = -s$ 
14:    end for
15:     $p_{watermark} = p_{watermark} \times p(d|watermark)$ 
16:  end if
17: end for
18: if  $p_{watermark} \geq \tau$  then
19:   return  $D'$ 
20: end if
21: return  $D$ 

```


Experiment 6 Malicious Datasets



中间模型访问、恶意采样、恶意改动数据集、协议4、工匠5、敌手3

Crafter 5 Malicious dataset

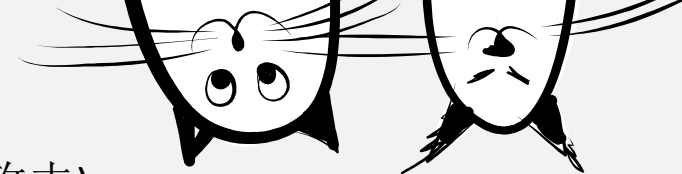
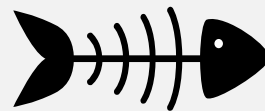
Require: initial model f_0 , dataset size n , input space $\mathcal{D}$, clipping norm C , selected params $2n$

- 1: $D_{data} \leftarrow$ randomly sample n random instances from $\mathcal{D}$
- 2: $D_{label} \leftarrow f_0(D_{data})$
- 3: $D \leftarrow (D_{data}, D_{label})$
- 4: **return** Call Crafter 4 on D

Distinguisher 3 Watermark detection

Require: subsampling rate q , number of iterations T , clipping norm C , number of poison parameters $2n$, number of measurements t_o , trained models $f_0, \dots, f_T$, threshold τ

- 1: $M \leftarrow \vec{0}$
- 2: $p_{watermark} \leftarrow 1$
- 3: **for** $t = 1$ to T **do**
- 4: **if** $t \leq t_o$ **then** 先训练 t_o 轮，用来找
- 5: $M = M + |f_t - f_{t-1}|$ 梯度最小的 $2n$ 个点
- 6: **else**
- 7: $points \leftarrow$ select smallest $2n$ arguments from M
- 8: $\nabla_{malicious} \leftarrow \vec{0}$
- 9: $s \leftarrow 1$
- 10: $d \leftarrow 0$
- 11: **for** p in $points$ **do** 这里是不是没写清楚，
- 12: $d = d + s \cdot f_t - f_{t-1}[p]$ $s \cdot [f_t[p] - f_{t-1}[p]]?$
- 13: $s = -s$
- 14: **end for**
- 15: $p_{watermark} = p_{watermark} \times p(d|watermark)$
- 16: **end if**
- 17: **end for**
- 18: **if** $p_{watermark} \geq \tau$ **then**
- 19: **return** D'
- 20: **end if**
- 21: **return** D



中间模型访问、自适应采样、不改动数据集、协议3、工匠3、敌手2(白盒成员推理攻击)

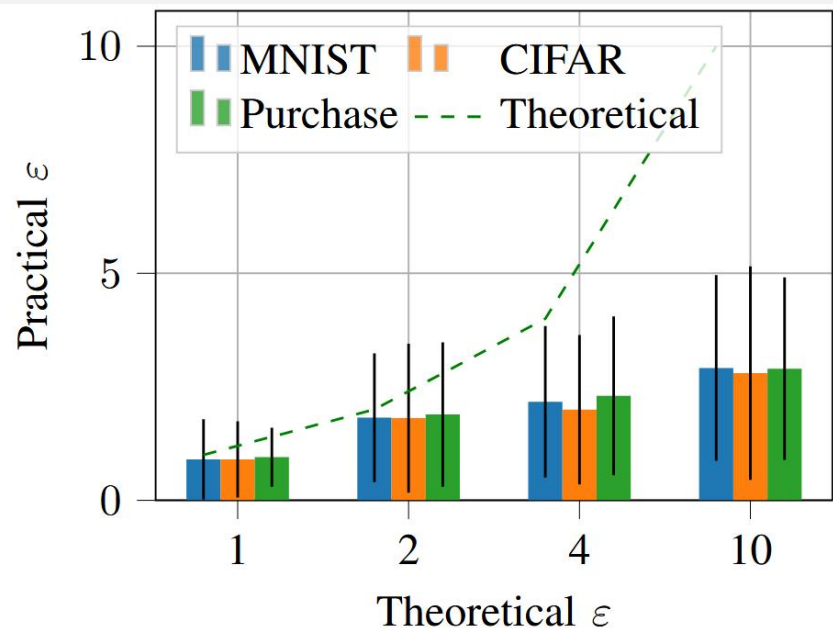


Fig. 7: **Gradient attack:** the adversary directly insert a malicious gradient to training. The results are consistently better than the previous attack.

与之前的对手相比，我们清楚地看到，消除由其他样本的梯度更新引起的“内在噪声”显著收紧了下界。我们得出结论，一个利用DP-SGD分析中所有假设的对手能够泄露接近最大值的私人信息。这表明，通过更好的DP-SGD分析实现“免费隐私”的时代已经结束，使用的是现有的假设。如果DP-SGD分析能够更好地捕捉典型数据集固有的噪声，可能就有可能实现更严格的保证——然而，即使是正式定义这种“内在噪声”意味着什么，也是一项非平凡的任务。



丸一口

感谢聆听



THANKS FOR YOUR WATCHING

